

12

Hot Dog or Not Hot Dog Using Machine Learning

In this chapter, we will learn how to use machine learning to create a model that we can use for image classification. We will export the model as an **Onnx** model that we can use on all platforms – that is, Android, iOS, macOS, and Windows. To train and export models, we will use Azure Cognitive Services and the Custom Vision service.

Once we have exported the models, we will learn how to use them in a .NET MAUI app.

The following topics will be covered in this chapter:

- Training a model with Azure Cognitive Services and the Custom Vision service
- Using Onnx models for image classification using ML.NET
- Using custom routes in .NET MAUI for navigation

Technical requirements

To be able to complete this project, you need to have Visual Studio for Mac or PC installed, as well as the .NET MAUI components. See *Chapter 1, Introduction to .NET MAUI*, for more details on how to set up your environment. You also need an Azure account. If you have a Visual Studio subscription, there are a specific amount of Azure credits included each month. To activate your Azure benefits, go to <https://my.visualstudio.com>.

You can also create a free account, where you can use selected services for free over 12 months. You will get \$200 worth of credit to explore any Azure service for 30 days, and you can also use the free services at any time. Read more at <https://azure.microsoft.com/en-us/free/>.

If you do not have and do not want to sign up for a free Azure account, the trained model is available in the source code for this chapter. You can download and use the pre-trained model instead.

The source code for this chapter is available at the GitHub repository for the book at <https://github.com/PacktPublishing/MAUI-Projects-3rd-Edition>.

Machine learning

The term **machine learning** was coined in 1959 by Arthur Samuel, an American pioneer in **artificial intelligence (AI)**. Tom M. Mitchell, an American computer scientist, provided the following more formal definition of machine learning later:

“A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P if its performance at tasks in T , as measured by P , improves with experience E .”

In simpler terms, this quote describes a computer program that can learn without being explicitly programmed. In machine learning, algorithms are used to build a mathematical model of sample data or training data. The models are used for computer programs to make predictions and decisions without being explicitly programmed for the task in question.

In this section, we will learn about a few different machine learning services and APIs that are available when developing a .NET MAUI application. Some APIs are only available for specific platforms, such as Core ML, while others are cross-platform.

Azure Cognitive Services – Custom Vision

Custom Vision is a tool or service that can be used to train models for image classification and to detect objects in images. With Custom Vision, we can upload our own images and tag them so that they can be trained for image classification. If we train a model for object detection, we can also tag specific areas of an image. Because models are already pre-trained for basic image recognition, we don't need a large amount of data to get a great result. The recommendation is to have at least 30 images per tag.

When we have trained a model, we can use it with an API, which is part of the Custom Vision service. We can also export models for **Core ML (iOS)**, **TensorFlow (Android)**, the **Open Neural Network Exchange (ONNX)**, and a **Dockerfile (Azure IoT Edge, Azure Functions, and Azure ML)**. These models can be used to carry out classification or object detection without being connected to the Custom Vision service.

You will need an Azure subscription to use it – go to <https://azure.com/free> to create a free subscription, which should be enough to complete this project.

Core ML

Core ML is a framework that was introduced in iOS 11. Core ML makes it possible to integrate machine learning models into iOS apps. On top of Core ML, we have high-level APIs, as follows:

- Vision APIs for image analysis
- Natural language APIs for natural language processing
- Speech to convert audio to text
- Sound analysis to identify sounds in audio
- GameplayKit to evaluate learned decision trees and strategies

MORE INFORMATION

More information about Core ML can be found in the official documentation from Apple at <https://developer.apple.com/documentation/coreml>.

TensorFlow

TensorFlow is an open source machine learning framework. However, TensorFlow can be used for more than simply running models on mobile devices – it can also be used to train models. To run it on mobile devices, we have TensorFlow Lite. The models that are exported from Azure Cognitive Services are for TensorFlow Lite. There are also C# bindings for TensorFlow Lite that are available as a NuGet package.

MORE INFORMATION

More information about TensorFlow can be found in the official documentation at <https://www.tensorflow.org/>.

ML.Net

ML.NET is an open source and cross-platform machine learning framework with support for iOS, macOS, Android, and Windows, all from a familiar environment – C#. ML.NET provides **AutoML**, a set of productivity tools that make building, training, and deploying custom models easy. ML.NET can be used in the following scenarios and more:

- Sentiment analysis and product recommendation
- Object detection and image classification
- Price prediction, sales spike detection, and forecasting
- Fraud detection
- Customer segmentation

Now that we have a broad overview of the technologies at play, we will focus on using ML.NET, since it is a cross-platform framework and built for C#. Let's look at the project we are going to build next.

The project overview

If you have seen the TV series *Silicon Valley*, you have probably heard of the *Not Hotdog* application. In this chapter, we will learn how to build that app. The first part of this chapter will involve collecting the data that we will use to create a machine learning model that can detect whether a photo contains a hot dog.

In the second part of the chapter, we will build an app using .NET MAUI and ML.NET, where the user can either take a new photo or pick a photo in the photo library, analyzing it to see whether it contains a hot dog. The estimated time for completing this project is 120 minutes.

Getting started

We can use either Visual Studio 2022 on a PC or Visual Studio for Mac to do this project. To build an iOS app using Visual Studio for PC, you must have a Mac connected. If you don't have access to a Mac at all, you can choose to just do the Android and Windows parts of this project.

Similarly, if you only have a Mac, you can choose to just do the iOS and macOS or Android parts of this project.

Building the Hot Dog or Not Hot Dog application using machine learning

Let's get started! We will first train a model for image classification that we can use later in the chapter to decide whether a photo contains a hot dog.

NOTE

If you do not want to go through the effort of training a model, you can download a pre-trained model from the following URL:

<https://github.com/PacktPublishing/MAUI-Projects-3rd-Edition/tree/main/Chapter12/HotdogOrNot/Resources/Raw>.

Training a model

To train a model for image classification, we need to collect photos of hot dogs and photos that aren't of hot dogs. Because most items in the world are not hot dogs, we need more photos that don't contain hot dogs. It's better if the photos of hot dogs cover a lot of different hot dog scenarios – with bread, ketchup, or mustard. This is so that the model will be able to recognize hot dogs in different situations. When we collect photos that aren't of hot dogs, we also need to have a large variety of photos that are both of items that are like hot dogs and that are completely different from hot dogs.

The model that is in the solution on GitHub was trained with 240 photos, 60 of which were of hot dogs, and 180 of which were not.

Once we have collected all the photos, we will be ready to start training the model by following these steps:

1. Go to <https://customvision.ai>.
2. Log in and create a new project.
3. Give the project a name – in our case, **HotDogOrNot**.
4. Select a resource or create a new one by clicking **Create new**. Fill in the dialog box, and select **CustomVision.Training** in the **Kind** dropdown.
The project type should be **Classification**, and the classification type should be **Multiclass (Single tag per image)**.
5. Select **General (compact)** as the domain. We use a compact domain if we want to export models and run them on a mobile device.
6. Click **Create project** to continue, as shown in the following screenshot:

Create new project

Name*
HotdogOrNot

Description
Enter project description

Resource*
HotDogOrNot [S0] [create new](#)
[Manage Resource Permissions](#)

Project Types ⓘ
☒ Classification
☐ Object Detection

Classification Types ⓘ
☐ Multilabel (Multiple tags per image)
☒ Multiclass (Single tag per image)

Domains:
☐ General [A2]
☐ General [A1]
☐ General
☐ Food
☐ Landmarks
☐ Retail
☐ General (compact) [S1]
☒ General (compact)
☐ Food (compact)
☐ Landmarks (compact)
☐ Retail (compact)

Pick the domain closest to your scenario. Compact domains are lightweight models that can be exported to iOS/Android and other platforms. [Learn More](#)

Export Capabilities: ⓘ
☒ Basic platforms (Tensorflow, CoreML, ONNX, ...)
☐ Vision AI Dev Kit

Cancel Create project

Figure 12.1 – Creating a new AI project

Once we have created a project, we can start to upload images and tag them.

Tagging images

The easiest way to get images is to go to Google and search for them. We will start by adding photos of hot dogs by following these steps:

1. Click **Add images**.
2. Select the photos of hot dogs that should be uploaded.
3. Tag the photos with **hotdog**, as shown in the following screenshot:

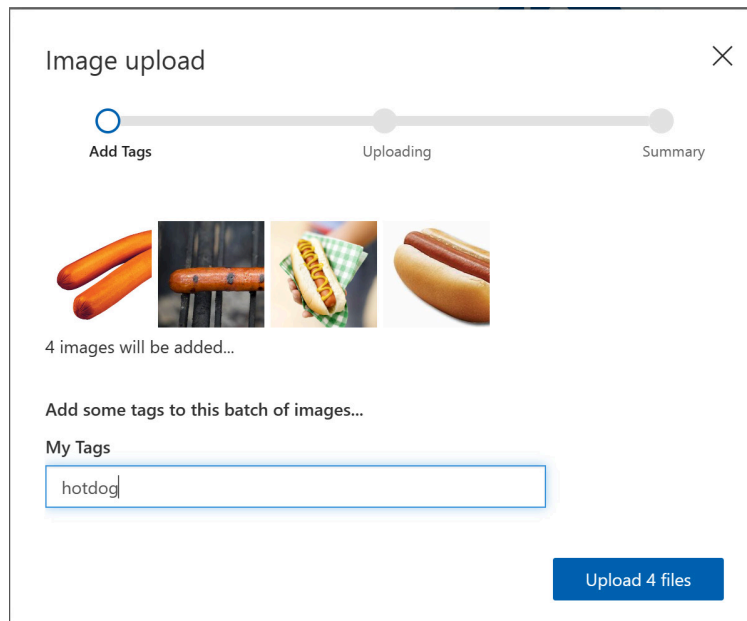


Figure 12.2 – Uploading images of hot dogs

Once we have uploaded all the photos of hot dogs, it's time to upload photos that aren't of hot dogs by following the following steps. For best results, we should also include photos of objects that look similar to hot dogs but are not:

1. Click the **Add images** button above the gallery of uploaded images.
2. Select the photos that aren't of hot dogs.
3. Tag the photos with **Negative**.

A Negative tag is used for photos that don't contain any objects that we have created other tags for. In this case, none of the photos we will upload contain hot dogs, as can be seen in the following screenshot:

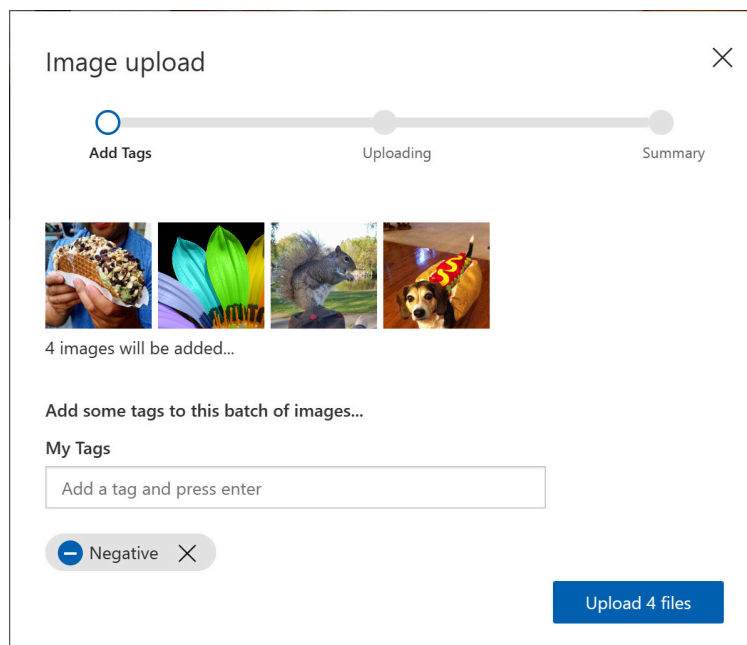


Figure 12.3 – Uploading images that aren't hot dogs

Once we have uploaded the photos, it's time to train a model.

Training a model

Not all the photos that we are uploading will be used for training; some will be used for verification to give us a score about how good the model is. If we upload photos in chunks and train the model after each chunk, we will be able to see our scores improving. To train a model, click the green **Train** button at the top of the page, as illustrated in the following screenshot:

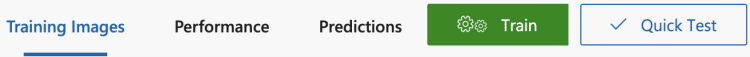


Figure 12.4 – Training the model

The following screenshot shows the result of a training iteration where the precision of the model is **91.7%**:

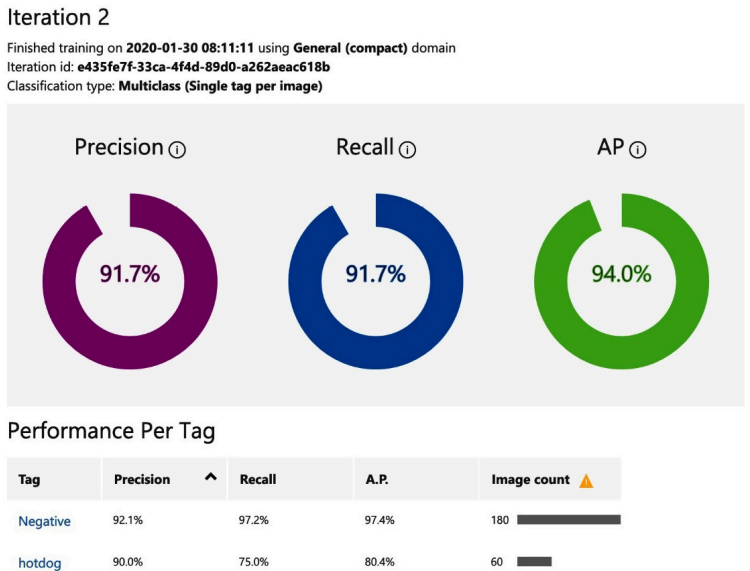


Figure 12.5 – Model verification results

Once we have trained a model, we will export it so that it can be used on a device.

Exporting a model

We can use the APIs if we want to, but to make fast classifications and to be able to do this offline, we will add the models to the app packages. Click the **Export** button and then on **ONNX** to download the model, as shown in the following screenshot:

Choose your platform

**CoreML**

iOS 11

**TensorFlow**

Android

**ONNX**

Windows ML

**Dockerfile**Azure IoT Edge, Azure Functions,
AzureML**Vision AI Dev Kit** ⓘ

Figure 12.6 – Exporting the model

Once we have downloaded the ONNX model, it's time to build the app.

Building the app

Our app will use the trained models to classify photos, according to whether they are photos of hot dogs. We will use the same ONNX model for all platforms in the .NET MAUI app.

Creating the new project

Let's begin, as follows.

The first step is to create a new .NET MAUI project:

1. Open Visual Studio 2022, and select **Create a new project**:

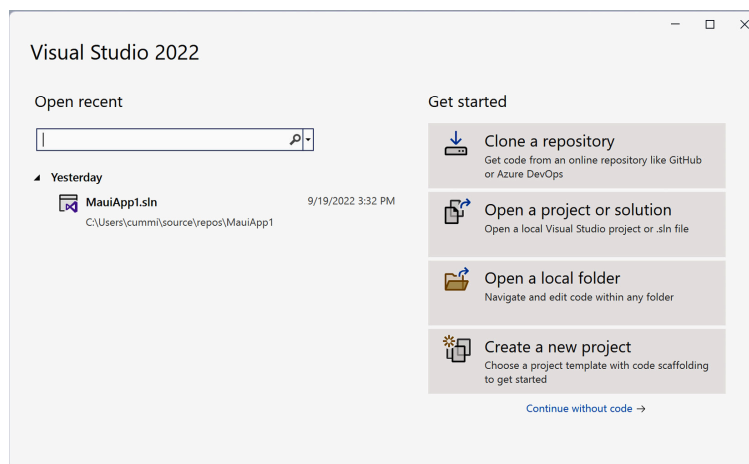


Figure 12.7 – Visual Studio 2022

This will open the **Create a new project** wizard.

2. In the search field, type in **maui**, and select the **.NET MAUI App** item from the list:

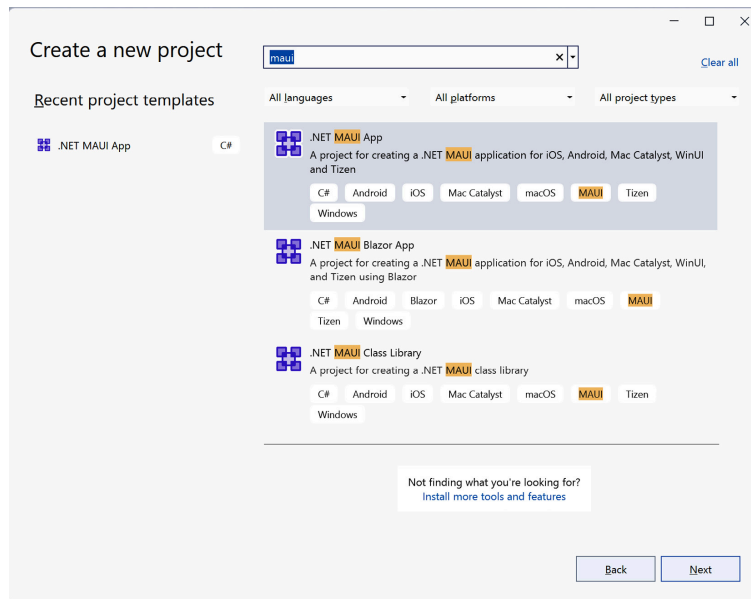


Figure 12.8 – Create a new project

3. Click **Next**.
4. Complete the next step of the wizard by naming your project. We will call our application **HotdogOrNot** in this case. Move on to the next dialog box by clicking **Next**, as illustrated in the following screenshot:

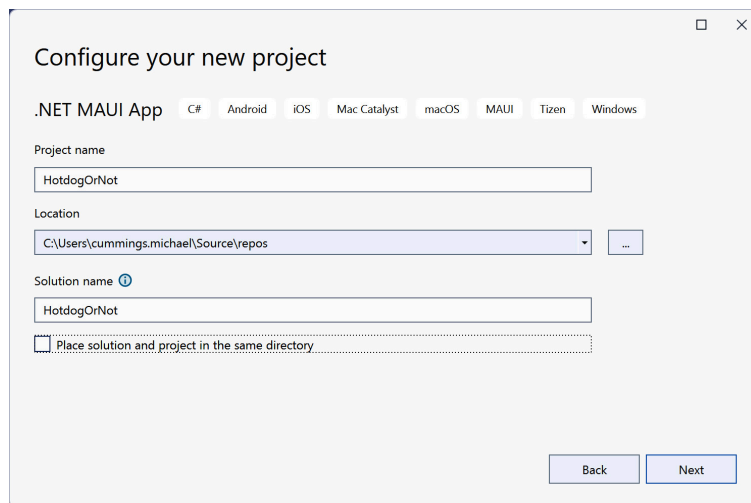


Figure 12.9 – Configure your new project

5. The last step will prompt you for the version of .NET Core to support. At the time of writing, .NET 6 is available as **Long-Term Support (LTS)**, and .NET 7 is available as **Standard Term Support**. For the purposes of this book, we will assume that you are using .NET 7.

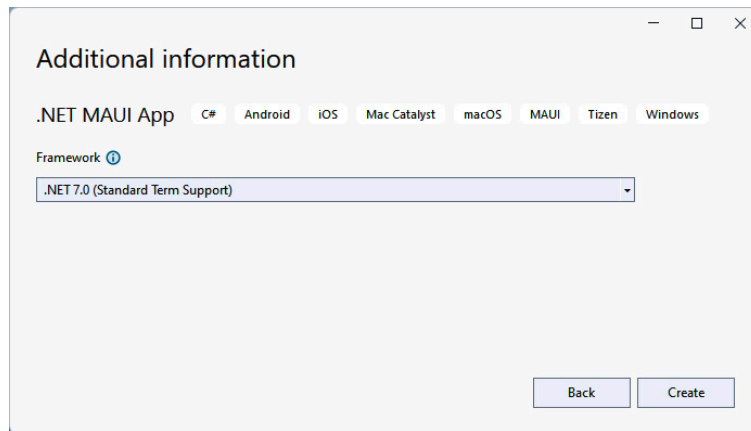


Figure 12.10 – Additional information

6. Finalize the setup by clicking **Create**, and wait for Visual Studio to create the project.

If you run the app now, you should see something like the following:

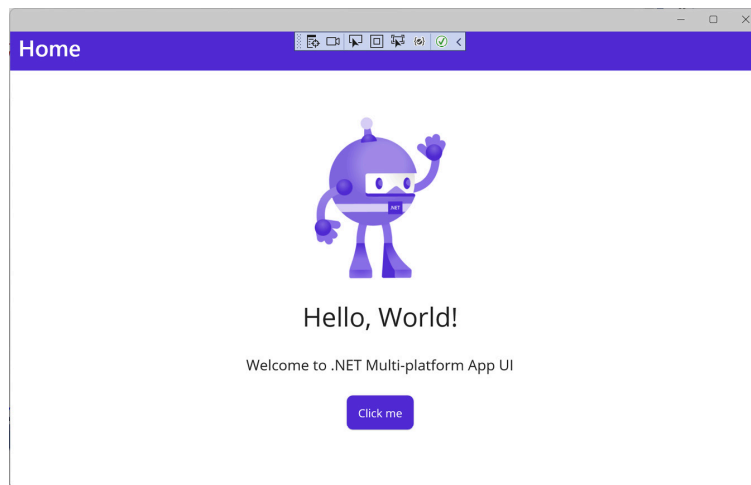


Figure 12.11 – The HotdogOrNot application

Just like that, the app is created. Next, let's start creating the image classifier.

Classifying images with machine learning

The first thing we will do is add the ONNX ML model to the project, by following these steps:

1. Extract the **.zip** file that we got from the Custom Vision service.
2. Find the **.onnx** file, and rename it **hotdog-or-not.onnx**.
3. Add it to the **Resources/Raw** folder in the project.

Once we add the file to the project, we are ready to create the implementation of the image classifier. The code that we will use for image classification will be shared between the .NET MAUI-supported platforms. We can create an interface for the classifier by following these steps:

1. Create a new folder named **ImageClassifier**.

2. Create a new class called **ClassifierOutput** in the **ImageClassifier** folder.
3. Modify the **ClassifierOutput** class to look like the following:

```
namespace HotdogOrNot.ImageClassifier;
internal sealed class ClassifierOutput
{
    ClassifierOutput() { }
}
```

4. Create a new interface called **IClassifier** in the **ImageClassifier** folder.
5. Add a method called **Classify** that returns **ClassifierOutput** and takes **byte[]** as an argument.
6. Your interface should look like the following code block:

```
namespace HotdogOrNot.ImageClassifier;
public interface IClassifier
{
    ClassifierOutput Classify(byte[] bytes);
}
```

Now that we have the interface for the classifier, we can move on to the implementation.

Using ML.NET for image classification

We are now ready to create the implementation of the **IClassifier** interface. Before we jump right into the implementation, let's take a look at the high-level steps that will need to happen so that we understand the flow a little better.

Our trained model, **hotdog-or-not.onnx**, has specific input and output parameters, and we will need to convert the image that we want to classify into the input format before submitting it to the ML.NET framework. Additionally, we need to ensure that the image is in the correct shape before submitting it. The shape of the image is defined by the size, width, height, and color format. If the image does not match the input format, then it needs to be resized and converted before submission, or you will run the risk of the image being classified incorrectly. For image classification models that are generated by the Custom Vision service, such as the *hotdog-or-not* model, the inputs and outputs look like the following:

INPUTS	
data	name: data
	type: float32[None, 3, 224, 224]
	denotation: Image(Bgr8,SRGB,NominalRange_0_255)
	Image(s) in BGR format. It is a [N, C, H, W]-tensor. The 1st/2nd/3rd slices along the C-axis are blue, green, and red channels, respectively.
OUTPUTS	
classLabel	name: classLabel
	type: string[None, 1]
loss	name: loss
	type: sequence<map<string, float32>>

Figure 12.12 – Model inputs and outputs from Netron

The input to the model is formatted into a multidimensional array named **data**. There are four dimensions that make up the array:

- **The image:** The format allows you to submit multiple images at once; however, for this app, we will only submit one image at a time
- **The color:** Each index in this dimension represents a single-color channel – **0** is blue, **1** is green, and **2** is red
- **The height:** Each index is a position along the y, or vertical, axis of an image, in the range between 0 and 223
- **The width:** Each index is a position along the x, or horizontal, axis of an image, in the range between 0 and 223

The value is the color value for that specific image, color, and x and y positions. For example, **data[0, 2, 64, 64]** would be the value of the green channel in the first image at a position of 64 pixels from the left and 64 pixels from the bottom of the image.

To reduce the number of incorrect classifications, we need to scale all the images for submission to 224 x 224 pixels and order the color channels properly.

We can do that by following these steps:

1. Create a new class called **MLNetClassifier** in the **ImageClassifier** folder of the project.
2. Add the **IClassifier** interface.
3. Implement the **Classify** method from the interface, as shown in the following code block:

```
namespace HotdogOrNot.ImageClassifier;
internal class MLNetClassifier : IClassifier
{
    public MLNetClassifier(byte[] model)
    {
        // Initialize Model here
    }
    public ClassifierOutput Classify(byte[] imageBytes)
    {
```

```
// Code will be added here

}

}
```

So far, we have not referenced any classes from ML.NET. To use the ML.NET APIs, we will need to add a reference to the NuGet package, by following these steps:

1. In the project, install the **Microsoft.ML.OnnxRuntime** NuGet package.
2. Accept any license dialog boxes.

This will install the relevant NuGet packages.

Now that we are referencing the ML.NET package, we can compile the ONNX ML model by following these steps:

1. At the top of the **MLNetClassifier** file, add the **using Microsoft.ML.Onnx Runtime;** declaration.
2. In the **MLNetClassifier** class, add the following fields:

```
readonly InferenceSession session;
readonly bool isBgr;
readonly bool isRange255;
readonly string inputName;
readonly int inputSize;
```

3. In the **MLNetClassifier** constructor, add the following lines of code to initialize the **OnnxRuntime** session, replacing the **// Initialize Model here** comment:

```
Session = new InferenceSession(model);
isBgr = session.ModelMetadata.CustomMetadataMap["Image.BitmapPixelFormat"] == "Bgr8";
isRange255 = session.ModelMetadata.CustomMetadataMap["Image.NominalPixelRange"] == "Nom";
inputName = session.InputMetadata.Keys.First();
inputSize = session.InputMetadata[inputName].Dimensions[2];
```

Let's discuss the preceding code before moving on. The constructor for the **MLNetClassifier** class accepts **byte[]** as a parameter. This represents the ML model file. **byte[]** is then passed into a new instance of **InferenceSession**, which is the main entry point into the ML.NET API. Once the model is loaded into the session, we can then inspect the model for certain properties, such as the image format (**isBGR**), the color value range (**isRange255**), the input name, and the input size. We cache these values in the class fields for use during classification. Your **MLNetClassifier** class should now look like the following:

```
using Microsoft.ML.OnnxRuntime;
namespace HotdogOrNot.ImageClassifier;
internal class MLNetClassifier : IClassifier
{
    readonly InferenceSession session;
    readonly bool isBgr;
    readonly bool isRange255;
    readonly string inputName;
    readonly int inputSize;
    public MLNetClassifier(byte[] model)
```

```

    {
        session = new InferenceSession(model);
        isBgr = session.ModelMetadata.CustomMetadataMap["Image.BitmapPixelFormat"] == "Bgr8";
        isRange255 = session.ModelMetadata.CustomMetadataMap["Image.NominalPixelRange"] == "Nom";
        inputName = session.InputMetadata.Keys.First();
        inputSize = session.InputMetadata[inputName].Dimensions[2];
    }
    public ClassifierOutput Classify(byte[] imageBytes)
    {
        // Code will be added here
    }
}

```

We can now move on to implementing the **Classify** method of the **MLNetClassifier** class.

The first step in running a classification is to get the input into the correct format. For image classification, that means resizing the image to the right dimensions and organizing the color values into the expected format. The image data is then loaded into **Tensor**, which is how we pass data into an ML.NET model. The following steps will create a method named **LoadInputTensor** to do just that:

1. Add a new method named **LoadInputTensor** after the **Classify** method in the **MLNetClassifier** class. This method will accept four parameters, **byte[]**, **int**, and two Booleans, and return a tuple of **Tensor<float>** and **byte[]**. Your method should look like the following:

```

static (Tensor<float>, byte[] resizedImage) LoadInputTensor(byte[] imageBytes, int imageSize, b
{
}

```

2. Inside **LoadInputTensor**, we will create the **return** objects and add the following highlighted lines of code:

```

{
    var input = new DenseTensor<float>(new[] { 1, 3, imageSize, imageSize });
    byte[] pixelBytes;
    // Add code here
    return (input, pixelBytes);
}

```

The next step is to resize the image; we will use the **ImageSharp** NuGet library to make this very easy.

3. Add the ImageSharp NuGet package to the project.
4. Add the following lines of code to resize the image, replacing the **// Add code here** comment:

```

using (var image = Image.Load<Rgb24>(imageBytes))
{
    image.Mutate(x => x.Resize(imageSize, imageSize));
    pixelBytes = new byte[image.Width * image.Height * Unsafe.SizeOf<Rgba32>()];
    image.ProcessPixelRows(source =>
    {
        // Add Code here
    }
    );
}

```

```
});
}
```

This code uses the ImageSharp library to load the image from `byte[]`. The image is then resized to the size required by the model. We use the `imageSize` field, whose value captures the model requirement from the constructor. Finally, we set up a call to the `ProcessPixelRows` method that will allow us to manipulate the individual pixels in the image.

5. Due to conflicts in naming between .NET MAUI and ImageSharp, we must add a declaration that tells the compiler which class we really want to use at the top of the file:

```
using Image = SixLabors.ImageSharp.Image;
```

6. The next section of code will also need the following highlighted declarations:

```
using Microsoft.ML.OnnxRuntime;
using SixLabors.ImageSharp.Formats.Png;
using Microsoft.ML.OnnxRuntime.Tensors;
using Image = SixLabors.ImageSharp.Image;
```

7. To get the input image into the correct color format required by the model, we use the `ProcessPixelRows` method from the `ImageSharp` library. This method provides a writable buffer for us to manipulate. Use the following highlighted code, in place of the `// Add Code here` comment, to iterate over the resized image data, putting the color values into the right order and clamping the values between 0 and 255, if required:

```
image.ProcessPixelRows(source =>
{
    for (int y = 0; y < image.Height; y++)
    {
        Span<Rgb24> pixelSpan = source.GetRowSpan(y);
        for (int x = 0; x < image.Width; x++)
        {
            if (isBgr)
            {
                input[0, 0, y, x] = pixelSpan[x].B;
                input[0, 1, y, x] = pixelSpan[x].G;
                input[0, 2, y, x] = pixelSpan[x].R;
            }
            else
            {
                input[0, 0, y, x] = pixelSpan[x].R;
                input[0, 1, y, x] = pixelSpan[x].G;
                input[0, 2, y, x] = pixelSpan[x].B;
            }
            if (!isRange255)
            {
                input[0, 0, y, x] = input[0, 0, y, x] / 255;
                input[0, 1, y, x] = input[0, 1, y, x] / 255;
                input[0, 2, y, x] = input[0, 2, y, x] / 255;
            }
        }
    }
}
```



```
    }
  });
```

What this code does is simple – using the provided source variable, it iterates over each row in the image, and each pixel in the row. If the model expects the colors to be in the blue, green, and red order, **isBGR** is **true**, and then the extracted color values are placed in the input tensor in that order; otherwise, they are added to the input tensor in the red, green, and blue order. The tricky part here is accessing the correct element for each pixel. The tensor is organized into four dimensions, as explained previously. The first element will always be zero for this model, since we are only processing one image at a time. The second dimension is the color channel, so you will see that change for the red, green, and blue color values.

Finally, if the model expects color values to be in the range of 0 to 255, **isRange255**, then each color channel is clamped to that range.

8. The last thing that we will do is copy the contents of the resized image to the **pixelBytes** array so that we can display the image to the user. Add the following highlighted code to do this; note that the previous code has been omitted for brevity:

```
    });
    var outputStream = new MemoryStream();
    image.Save(outputStream, new PngEncoder());
    pixelBytes = outputStream.ToArray();
  }
  return (input, pixelBytes);
```

Now that we have written the code to process the image and populate the input tensor, we can complete the **Classify** method by following these steps:

1. Replace the **// Code will be added here** comment with a call to the **LoadInputTensor** method:

```
public ClassifierOutput Classify(byte[] imageBytes)
{
    (Tensor<float> tensor, byte[] resizedImage) = LoadInputTensor(imageBytes, inputSize, is
```

2. Next, we can run the session, passing in the newly created input tensor and capturing the result:

```
public ClassifierOutput Classify(byte[] imageBytes)
{
    (Tensor<float> tensor, byte[] resizedImage) = LoadInputTensor(imageBytes, inputSize, is
    var resultsCollection = session.Run(new List<NamedOnnxValue>
    {
        NamedOnnxValue.CreateFromTensor<float>(inputName, tensor)
    });
}
```

3. We grab the label from the output result, which will be used to determine whether this image contains a hotdog or not:

```

public ClassifierOutput Classify(byte[] imageBytes)
{
    (Tensor<float> tensor, byte[] resizedImage) = LoadInputTensor(imageBytes, inputSize, is
    var resultsCollection = session.Run(new List<NamedOnnxValue>
        {
            NamedOnnxValue.CreateFromTensor<float>(inputName, tensor)
        });
    var topLabel = resultsCollection
        ?.FirstOrDefault(i => i.Name == "classLabel")
        ?.AsTensor<string>()
        ?.First();
}

```

4. Then, we can get the confidence level of the result, which tells us how sure the model is of the classification. This will be used when we display the result:

```

public ClassifierOutput Classify(byte[] imageBytes)
{
    (Tensor<float> tensor, byte[] resizedImage) = LoadInputTensor(imageBytes, inputSize, is
    var resultsCollection = session.Run(new List<NamedOnnxValue>
        {
            NamedOnnxValue.CreateFromTensor<float>(inputName, tensor)
        });
    var topLabel = resultsCollection
        ?.FirstOrDefault(i => i.Name == "classLabel")
        ?.AsTensor<string>()
        ?.First();
    var labelScores = resultsCollection
        ?.FirstOrDefault(i => i.Name == "loss")
        ?.AsEnumerable<NamedOnnxValue>()
        ?.First()
        ?.AsDictionary<string, float>();
}

```

5. Finally, we can return the result of the classification using the **ClassifierOutput** class:

```

public ClassifierOutput Classify(byte[] imageBytes)
{
    (Tensor<float> tensor, byte[] resizedImage) = LoadInputTensor(imageBytes, inputSize, is
    var resultsCollection = session.Run(new List<NamedOnnxValue>
        {
            NamedOnnxValue.CreateFromTensor<float>(inputName, tensor)
        });
    var topLabel = resultsCollection
        ?.FirstOrDefault(i => i.Name == "classLabel")
        ?.AsTensor<string>()
        ?.First();
    var labelScores = resultsCollection
        ?.FirstOrDefault(i => i.Name == "loss")
        ?.AsEnumerable<NamedOnnxValue>()
        ?.First()
        ?.AsDictionary<string, float>();
    return ClassifierOutput.Create(topLabel, labelScores, resizedImage);
}

```

6. The last step is to finish the **MLNetClassifier** implementation by implementing the **ClassifierOutput** class. Update your

ClassifierOutput class by adding the highlighted code:

```
internal sealed class ClassifierOutput
{
    public string TopResultLabel { get; private set; }
    public float TopResultScore { get; private set; }
    public IDictionary<string, float> LabelScores { get; private set; }
    public byte[] Image { get; private set; }
    ClassifierOutput() { }
    public static ClassifierOutput Create(string topLabel, IDictionary<string, float> labelScores, byte[] image)
    {
        var topLabelValue = topLabel ?? throw new ArgumentException(nameof(topLabel));
        var labelScoresValue = labelScores ?? throw new ArgumentException(nameof(labelScores));
        return new ClassifierOutput
        {
            TopResultLabel = topLabelValue,
            TopResultScore = labelScoresValue.First(i => i.Key == topLabelValue).Value,
            LabelScores = labelScoresValue,
            Image = image,
        };
    }
}
```

The **ClassifierOutput** class is used to encapsulate the four values that will be used in the UI and expose them as public properties. The **Create** static method is used to create an instance of the class. The **Create** method validates the arguments provided and sets the public properties appropriately for use by the UI.

We have now written the code to recognize hot dogs in an image.

Now, we can build the user interface for the application and call **MLNetClassifier** to classify an image.

Requesting app permissions

Before we dive right into building the rest of the app functionality, we need to address permissions. This app will have two buttons that the user will use, one to take a photo and another to select a photo from the device. This is similar to the functionality that we saw in *Chapter 6, Building a Photo Gallery App Using CollectionView and CarouselView*, where we needed to request permission from the user before accessing the camera or device storage. However, we will implement the permissions differently than we did in that chapter. Since gaining access to the camera and accessing photos on the user's device requires separate permissions, we will request them from each button handler.

Follow these steps to add a class to help us with the permission checks:

1. Create a new class named **AppPermissions** in the project.
2. Modify the class definition to add a **partial** modifier, and remove the default constructor:

```
namespace HotdogOrNot;
internal partial class AppPermissions
```

```
{
}
```

3. Add the following method to the **AppPermissions** class:

```
public static async Task<PermissionStatus> CheckRequiredPermission<TPermission>() where TPe
```

The **CheckRequiredPermission** method is used to ensure that our app has the right permissions before we attempt any operations that might fail if we don't. Its implementation is to call the .NET MAUI **CheckSyncStatus** with the provided permission type in **TPermission**. It returns **PermissionStatus**, which is **enum**. We are mostly interested in the **Denied** and **Granted** values.

4. Add the **CheckAndRequestRequiredPermission** method to the **AppPermissions** class:

```
public static async Task<PermissionStatus> CheckAndRequestRequiredPermission() <TPermission
{
    PermissionStatus status = await Permissions.CheckStatusAsync< TPermission >();
    if (status == PermissionStatus.Granted)
        return status;
    if (status == PermissionStatus.Denied && DeviceInfo.Platform == DevicePlatform.iOS)
    {
        // Prompt the user to turn on in settings
        // On iOS once a permission has been denied it may not be requested again from the
        await App.Current.MainPage.DisplayAlert("Required App Permissions", "Please enable
    }
    if (Permissions.ShouldShowRationale< TPermission >())
    {
        // Prompt the user with additional information as to why the permission is needed
        await App.Current.MainPage.DisplayAlert("Required App Permissions", "This app uses
    }
    status = await MainThread.InvokeOnMainThreadAsync(Permissions.RequestAsync<TPermission>
    return status;
}
```

The **CheckAndRequestRequiredPermission** method handles the intricacies of requesting access from the user. The first step is to simply check and see whether the permission has already been granted and, if it has, return the status. Next, if we are on iOS and the permission has been denied, it cannot be requested again, so you must instruct the user on how to grant permission to the app by using the settings panel. Android includes in the request behavior the ability to nag the user if they have denied access. This behavior is exposed through .NET MAUI with the **ShouldShowRationale** method. It will return **false** for any platform that does not support this behavior, and on Android, it will return **true** the first time after the user denies access and **false** if the user denies it a second time. Finally, we request access to the permission from the user. Again, .NET MAUI hides all the platform implementation details from us, making checking and requesting access to certain resources very straightforward.

LOOK FAMILIAR?

If the preceding code looks familiar, then you are right. It is based on the implementation that is described in the .NET MAUI documentation. You can find it at <https://learn.microsoft.com/en-us/dotnet/maui/platform-integration/appmodel/permissions>.

Now that we have the shared **AppPermissions** in place, we can start with the platform configuration. Before we can use the media picker, however, we need to do some configuration for each platform. We will start with Android.

In Android API version 33, three new permissions were added to enable read access to media files – **ReadMediaImages**, **ReadMediaVideos**, and **ReadMediaAudio**. Prior to API version 33, all that was required was the **ReadExternalStorage** permission. To access the camera, we will need both **Camera** and **WriteExternalStorage** permissions. To properly request the correct permission for the API version of the device, open **MauiApplication.cs** in the **Platform/Android** folder and modify it to look like the following:

```
using Android.App;
using Android.Runtime;
// Needed for Picking photo/video
[assembly: UsesPermission(Android.Manifest.Permission.ReadExternalStorage, MaxSdkVersion = 32)]
[assembly: UsesPermission(Android.Manifest.Permission.ReadMediaImages)]
// Needed for Taking photo/video
[assembly: UsesPermission(Android.Manifest.Permission.Camera)]
[assembly: UsesPermission(Android.Manifest.Permission.WriteExternalStorage)]
```

Finally, for Android, we need to declare usage of the **IMAGE_CAPTURE** intent as follows in the **AndroidManifest.xml** file:

```
<queries>
  <intent>
    <action android:name="android.media.action.IMAGE_CAPTURE" />
  </intent>
</queries>
```

For iOS and Mac Catalyst, the only thing we need to do is add the following four usage descriptions to the **info.plist** file in the **platform/ios** and **platform/maccatalyst** folders:

```
<key>NSCameraUsageDescription</key>
<string>This app needs access to the camera to take photos.</string>
<key>NSPhotoLibraryUsageDescription</key>
<string>This app needs access to photos.</string>
<key>NSMicrophoneUsageDescription</key>
<string>This app needs access to microphone.</string>
<key>NSPhotoLibraryAddUsageDescription</key>
<string>This app needs access to the photo gallery.</string>
```

For Windows, we need to add the following highlighted code to the **Capabilities** section of the **package.appxmanifest** file in the **platforms/windows** folder:

```
<Capabilities>
  <rescap:Capability Name="runFullTrust" />
  <DeviceCapability Name="webcam"/>
</Capabilities>
```

Now that we have declared the permissions we need for each platform, we can implement the remaining functionality to take a photo or pick an existing image.

Building the first view

The first view in this app will be a simple view with two buttons. One button will be to start the camera so that users can take a photo of something to determine whether it is a hot dog. The other button will be to pick a photo from the photo library of the device. We will continue to use the MVVM pattern in this chapter, so we will split the view into two classes, **MainView** for the UI visible to the user and **MainViewModel** for the actual implementation.

Building the ViewModel class

We will start by creating the **MainViewModel** class, which will handle what will happen when a user taps one of the buttons. Let's set this up by going through the following steps:

1. Create a new folder called **ViewModels**.
2. Add a NuGet reference to **CommunityToolkit.Mvvm**; we use **CommunityToolkit.Mvvm** to implement the **INotifyPropertyChanged** interface and commands, as we did in other chapters.
3. Create a new partial class called **MainViewModel** in the **ViewModels** folder, using **ObservableObject** from the **CommunityToolkit.Mvvm.ComponentModel** namespace as a base class.
4. Create a private field of the **IClassifier** type and call it **classifier**, as shown in the following code block:

```
using CommunityToolkit.Mvvm.ComponentModel;
using HotdogOrNot.ImageClassifier;
namespace HotdogOrNot.ViewModels;
public partial class MainViewModel : ObservableObject
{
    private IClassifier classifier;
    public MainViewModel()
    {
    }
}
```

Initializing the ONNX model requires the use of asynchronous methods, so we need to handle them carefully, since we will be calling them from the constructor and the button handlers. The following steps will create the model initializer:

1. Create an **InitTask** property that is of the **Task** type.
2. Use a property initializer to set it to a new **Task**, using **Task.Run**.

3. Initialize the model from the raw resources of the .NET MAUI app. The method should look like the following code:

```
Task InitTask() => Task.Run(async () =>
{
    using var modelStream = await FileSystem.OpenAppPackageFileAsync("hotdog-or-not.onnx");
    using var modelMemoryStream = new MemoryStream();
    modelStream.CopyTo(modelMemoryStream);
    var model = modelMemoryStream.ToArray();
    _classifier = new MLNetClassifier(model);
});
```

The **InitTask** property holds a reference to **Task** that does the following:

1. Loads the **hotdog-or-not.onnx** file into **Stream**
2. Copies the bytes from the original stream to an array of bytes so that the original stream can be closed and any native resources, such as file handles, can be released.
3. Creates and returns a new instance of the **MLNetClassifier** class using the loaded model.
4. To ensure that **InitTask** will only run successfully once, add the following highlighted code:

```
public partial class MainViewModel : ObservableObject
{
    IClassifier _classifier;
    Task initTask;
    public MainViewModel()
    {
        _ = InitAsync();
    }
    public Task InitAsync()
    {
        if (initTask == null || initTask.IsFaulted)
            initTask = InitTask();
        return initTask;
    }
    // Code omitted for brevity
}
```

In **InitAsync**, the initialization task is captured by a field only if the field is **null** or its value has faulted. This ensures that we only run the initialization successfully once. The value of the field is then returned to the caller, which, in this case, is the constructor. Unwinding this, the constructor calls **InitAsync** and throws away the return value.

InitAsync, meanwhile, captures the value returned by the **InitTask** property, which is **Task** that has already been queued for execution. Since **InitAsync** and **InitTask** and their closure are all asynchronous, they complete sometime after the constructor completes.

Now that we have initialized the **hotdog-or-not** ONNX model, we can now implement the two buttons, one that takes a photo and another that allows the user to pick a photo from their device storage. Let's start by implementing a couple of helper methods to use in both use cases.

The first helper method is used to convert **FileResult** to **byte[]**. To implement **ConvertPhotoToBytes**, follow these steps:

1. Open the **MainViewModel.cs** file.
2. Add a new method named **ConvertPhotoToBytes**, which takes **FileResult** as a parameter and returns **byte []**. Since the method is **async**, you'll need to return **Task** and use the **async** modifier.
3. In the method, check whether **FileResult** is **null** and that it returns an empty array.
4. Next, open a stream from **FileResult** using the **OpenStreamAsync** method.
5. Create a new variable of the **MemoryStream** type and initialize it using the default constructor.
6. Use the **Copy** method to copy **stream** to **MemoryStream**.
7. Finally, return **MemoryStream** as **byte[]**; your method should look like the following:

```
private async Task<byte[]> ConvertPhotoToBytes(FileResult photo)
{
    if (photo == null) return Array.Empty<byte>();
    using var stream = await photo.OpenReadAsync();
    using MemoryStream memoryStream = new();
    stream.CopyTo(memoryStream);
    return memoryStream.ToArray();
}
```

The other helper method we will need is to use our classification model to get the results of a photo and return the results. We will need a new type to return the results. Follow these steps to implement the new class:

1. Create a new folder named **Models** in the project.
2. In the **Models** folder, create a new class, **Result**, in a file named **Result.cs**.
3. Add a public property, **IsHotdog**, as **bool**.
4. Add a public property, **Confidence**, as **float**.
5. Add a public property, **PhotoBytes**, as **byte[]**; the class should now look like the following:

```
namespace HotdogOrNot.Models;
public class Result
{
    public bool IsHotdog { get; set; }
    public float Confidence { get; set; }
    public byte[] PhotoBytes { get; set; }
}
```

The **IsHotdog** property is used to capture whether the label returned from the model is “hotdog.” **Confidence** is a score of how sure the model is that this is a hotdog or not. Finally, since we transform the image prior to processing, we store the transformed image in the **PhotoBytes** property.

Now, we can implement the method that will run and process the classification result, by following these steps:

1. Open the `MainViewModel.cs` file.
2. In the `MainViewModel` class, add a new field, `isClassifying`, with a `bool` type.
3. Add the `ObservableAttribute` attribute to the field; it should look like the following:

```
[ObservableProperty]
private bool isClassifying;
```

4. Add a new method to the `MainViewModel` class, named `RunClassificationAsync`. The method will accept a `byte[]` parameter and return `Result`, wrapped in `Task`, since it is `async`:

```
async Task<Result> RunClassificationAsync(byte[] imageToClassify)
{
}
```

5. In the method, the first thing we do is set the `IsClassifying` property to `true`; this will be used to disable the buttons later in the chapter.
6. Add a `try..catch..finally` statement.
7. Inside the `try` statement, ensure the model is initialized by calling `InitAsync`.
8. Then, call `Classify` on the `classifier` field passing `byte[]`, representing the image as a parameter and storing the result.
9. The last statement in the `try` statement block is to return a new `Result`, setting `IsHotdog` to `true` only if the classification result's `TopResultLabel` property is "hotdog," `Confidence` is set to the classification result's `TopResultScore` property, and `PhotoBytes` is set to the classification result's `Image` property. The `try` portion should look like the following:

```
try
{
    await InitAsync().ConfigureAwait(false);
    var result = _classifier.Classify(imageToClassify);
    return new Result()
    {
        IsHotdog = result.TopResultLabel == "hotdog",
        Confidence = result.TopResultScore,
        PhotoBytes = result.Image
    };
}
catch
```

10. Now, in the `catch` statement block, return a new `Result`, setting the `IsHotdog` property to `false`, `Confidence` to `0.0f`, and the `PhotoBytes` property to the bytes passed into the method. The `catch` block should look like the following:

```
catch
{
    return new Result
    {
        IsHotdog = false,
        Confidence = 0.0f,
```

```

        PhotoBytes = imageToClassify
    };
}
finally

```

11. Lastly, for the **finally** block, we want to set the **IsClassifying** property back to **false**; however, we will need to do this on the main UI thread using the **MainThread.BeginInvokeOnMainThread** method from .NET MAUI, as shown in the following code:

```

finally
{
    MainThread.BeginInvokeOnMainThread(() => IsClassifying = false);
}

```

Now that we have written the helper methods, we can create two methods, one to handle capturing an image from the camera and another to pick a photo from user storage. We will start with the camera capture method.

Let's set this up by following these steps:

1. Open the **MainViewModel.cs** file.
2. Create a public async void method called **TakePhoto**.
3. Add the **RelayCommand** attribute to make the method bindable.
4. Add an **if** statement to check whether the **MediaPicker.Default.IsCaptureSupported** parameter is **true**.
5. In the **true** statement block of **if**, get the status of the **Camera** permission using the **CheckAndRequestPermission** method.
6. If the status is **Granted**, then use **CheckAndRequestMethod** again to check the **WriteExternalStorage** permission.
7. If the status is **Granted**, use **MediaPicker** to capture a photo using the **CapturePhotoAsync** method.
8. Call a method named **ConvertPhotoToBytes**, passing in the file returned from **MediaPicker**.
9. Pass the photo bytes to the **RunClassificationAsync** method.
10. Finally, we will dynamically navigate to the **Result** view, which we will create in the next section, passing the result from **RunClassificationAsync** as a parameter. We do this by using **Shell.Current.GotoAsync** and ensuring that the app uses the main thread to do so, as shown in the following code block:

```

[RelayCommand()]
public async void TakePhoto()
{
    if (MediaPicker.Default.IsCaptureSupported)
    {
        var status = await AppPermissions.CheckAndRequestRequiredPermissionAsync<Permission>
        if (status == PermissionStatus.Granted) {
            status = await AppPermissions.CheckAndRequestRequiredPermissionAsync<Permission>
        }
        if (status == PermissionStatus.Granted)
        {
            FileResult photo = await MediaPicker.Default.CapturePhotoAsync(new MediaPickerO

```

```

        var imageToClassify = await ConvertPhotoToBytes(photo);
        var result = await RunClassificationAsync(imageToClassify);
        await MainThread.InvokeOnMainThreadAsync(async () => await
            Shell.Current.GoToAsync("Result", new Dictionary<string, object>() { { "res
        });
    }
}
}

```

Shell.Current.GoToAsync takes two parameters – the first is the route that **Shell** is to navigate to, and the second is a dictionary of key-value pairs to send to the destination view. Later in this chapter, we will see how to configure a route to a view without using XAML and, when we create the **Result** view, how to access the parameters passed to it.

We will now create the **PickPhoto** method to allow a user to use an image from their device. Use the following steps to create the method:

1. Create a public async void method called **PickPhoto**.
2. Add the **RelayCommand** attribute to make the method bindable.
3. Grant the status of the **Photos** permission using the **CheckAndRequestPermission** method.
4. If the status is **Granted**, use **MediaPicker** to capture a photo using the **Pick PhotoAsync** method.
5. Call a method named **ConvertPhotoToBytes**, passing in the file returned from **MediaPicker**.
6. Pass the photo bytes to the **RunClassificationAsync** method.
7. Finally, we will dynamically navigate to the **Result** view, which we will create in the next section, passing the result from **RunClassificationAsync** as a parameter. We will do this by using **Shell.Current.GoToAsync** and ensuring that the app uses the main thread to do so, as shown in the following code block:

```

[RelayCommand()]
public async void PickPhoto()
{
    var status = await AppPermissions.CheckAndRequestRequiredPermissionAsync<Permissions.Ph
    if (status == PermissionStatus.Granted)
    {
        FileResult photo = await MediaPicker.Default.PickPhotoAsync();
        var imageToClassify = await ConvertPhotoToBytes(photo);
        var result = await RunClassificationAsync(imageToClassify);
        await MainThread.InvokeOnMainThreadAsync(async () => await
            Shell.Current.GoToAsync("Result", new Dictionary<string, object>() { { "result"
        });
    }
}

```

When a user clicks on a button, the classification could take a noticeable amount of time. To prevent the user from clicking the button again because they think it's not working, we will disable the buttons until the operation completes. The **IsClassifying** property is already set; we just need to use that value to restrict **RelayCommands**, by following these steps:

1. Add a new method that returns a Boolean named **CanExecuteClassification**, and return the inverse of the **IsClassifying** property, as shown in the following code:

```
private bool CanExecuteClassification() => !IsClassifying;
```

2. Update the **RelayCommand** attribute for the **TakePhoto** method, as highlighted here:

```
[RelayCommand(CanExecute = nameof(CanExecuteClassification))]  
public async void TakePhoto()
```

3. Update the **RelayCommand** attribute for the **PickPhoto** method, as highlighted here:

```
[RelayCommand(CanExecute = nameof(CanExecuteClassification))]  
public async void PickPhoto()
```

Now that ViewModel for the main page is complete, we can build View for the main page.

Building the view

Now, once we have created the **MainViewModel** class, it is time to create the code for the **graphical user interface (GUI)**. Go through the following steps to create the GUI for the **MainView** view:

1. Create a new folder called **Views**.
2. Add a new **.NET MAUI ContentPage (XAML)** called **MainView**.
3. Set the **Title** property of **ContentPage** as **Hotdog or Not hotdog**.
4. Add **HorizontalStackLayout** to the page, and set its **VerticalOptions** property to **Center** and its **HorizontalOptions** property to **CenterAndExpand**.
5. Add **Button** to the **HorizontalStackLayout**, with the text **Take Photo**. For the **Command** property, add a binding to the **TakePhoto** property in the **MainViewModel** class.
6. Add **Button** to **HorizontalStackLayout**, with the text **Pick Photo**. For the **Command** property, add a binding to the **PickPhoto** property in the **MainViewModel** class, as shown in the following code block:

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"  
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"  
             xmlns:viewModels="clr-namespace:HotdogOrNot.ViewModels"  
             x:Class="HotdogOrNot.Views.MainView"  
             x:DataType="viewModels:MainViewModel"  
             Title="Hotdog or Not hotdog">  
    <HorizontalStackLayout VerticalOptions="Center" HorizontalOptions="CenterAndExpand">  
        <Button Text="Take Photo" Command="{Binding TakePhotoCommand}" WidthRequest="150" HeightRequest="40">  
        <Button Text="Pick Photo" Command="{Binding PickPhotoCommand}" WidthRequest="150" HeightRequest="40">  
    </HorizontalStackLayout>  
</ContentPage>
```

In the code-behind **MainView.xaml.cs** file, we will set the binding context of the view by following these steps:

1. Add **MainViewModel** as a parameter of the constructor.
2. After the **InitializeComponent** method call, set the **BindingContext** property of the view to the **MainViewModel** parameter.
3. Use the **SetBackButtonTitle** static method on the **NavigationPage** class so that an arrow to navigate back to this view will be shown in the navigation bar on the result view, as shown in the following code block:

```
public MainView(MainViewModel viewModel)
{
    InitializeComponent();
    BindingContext = viewModel; NavigationPage.SetBackButtonTitle(this, string.Empty);
}
```

Building the result view

The last thing we need to do in this project is to create the result view. This view will show the input photo and the classification of a hot dog or not.

Building the ResultViewModel class

Before we create the view, we will create a **ResultViewModel** class that will handle all the logic for the view, by following these steps:

1. Create a **partial** class called **ResultViewModel** in **ViewModels**.
2. Add **ObservableObject** as a base class to the **ResultViewModel** class.
3. Create a **private** field of the **string** type, called **title**. Add the **ObservableProperty** attribute to the field to make it a bindable property.
4. Create a **private** field of the **string** type, called **description**. Add the **ObservableProperty** attribute to the field to make it a bindable property.
5. Create a **private** field of the **string** type, called **Title**. Add the **ObservableProperty** attribute to the field to make it a bindable property, as shown in the following code block:

```
using CommunityToolkit.Mvvm.ComponentModel;
using HotdogOrNot.Models;
namespace HotdogOrNot.ViewModels;
public partial class ResultViewModel : ObservableObject
{
    [ObservableProperty]
    private string title;
    [ObservableProperty]
    private string description;
    [ObservableProperty]
    byte[] photoBytes;
    public ResultViewModel()
    {
    }
}
```

The next thing we will do in **ResultViewModel** is to create an **Initialize** method that will have the result as a parameter. Let's set this up by following these steps:

1. Add a **private** method named **Initialize** to the **ResultViewModel** class that accepts a parameter of the **Result** type, named **result**, and returns **void**.
2. In the **Initialize** method, set the **PhotoBytes** property to the value of the **PhotoBytes** property of the **result** parameter.
3. Add an **if** statement that checks whether the **IsHotDog** property of the **result** parameter is **true** and whether **Confidence** is higher than **90%**. If this is the case, set **Title** to **"Hot dog"** and **Description** to **"This is for sure a hotdog"**.
4. Add an **else if** statement to check whether the **IsHotdog** property of the **result** parameter is **true**. If this is the case, set **Title** to **"Maybe"** and **Description** to **"This is maybe a hotdog"**.
5. Add an **else** statement that sets **Title** to **"Not a hot dog"** and **Description** to **"This is not a hot dog"**, as shown in the following code block:

```
public void Initialize(Result result)
{
    PhotoBytes = result.PhotoBytes;
    if (result.IsHotdog && result.Confidence > 0.9)
    {
        Title = "Hot dog";
        Description = "This is for sure a hot dog";
    }
    else if (result.IsHotdog)
    {
        Title = "Maybe";
        Description = "This is maybe a hot dog";
    }
    else
    {
        Title = "Not a hot dog";
        Description = "This is not a hot dog";
    }
}
```

The final thing we need to do is call the **Initialize** method with the result. If you recall from the previous section on building the main view, we navigated to the **Result** view and passed the **Result** object as a parameter. To access the parameter and call the **Initialize** method properly, follow these steps:

1. Add the **IQueryAttributable** interface to the list of inherited interfaces:

```
public partial class ResultViewModel : ObservableObject, IQueryAttributable
```

2. Add a new **void** method, **ApplyQueryAttributes**, that accepts a parameter named **query** of the **IDictionary<string, object>** type:

```
public void ApplyQueryAttributes(IDictionary<string, object> query)
{
}
```

3. Now, in the method, call the **Initialize** method, passing the “**result**” object from the query dictionary and casting it to a **Result** type, as shown in the following code:

```
public void ApplyQueryAttributes(IDictionary<string, object> query)
{
    Initialize(query["result"] as Result);
}/
```

ViewModel is now complete, and we are ready to create **View**.

Building the view

Because we want to show the input photo in the result view, we need to convert it from **byte[]** to **Microsoft.Maui.Controls.ImageSource**. We will do this in a value converter that we can use together with the binding in the **XAML**, by following these steps:

1. Create a new folder called **Converters**.
2. Create a new class called **BytesToImageConverter** in the **Converters** folder.
3. Add and implement the **IValueConverter** interface, as shown in the following code block:

```
using System.Globalization;
namespace HotdogOrNot.Converters;
public class BytesToImageConverter : IValueConverter
{
    public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
    {
        throw new NotImplementedException();
    }
    public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
    {
        throw new NotImplementedException();
    }
}
```

The **Convert** method will be used when **ViewModel** updates a view. The **ConvertBack** method will be used in two-way bindings when **View** updates **ViewModel**. In this case, we only need to write code for the **Convert** method, by following these steps:

1. First, check whether the **value** parameter is **null**. If so, we should return **null**.
2. If the value is not **null**, cast it as **byte[]**.
3. Create a **MemoryStream** object from the **byte** array.
4. Return the result of the **ImageSource.FromStream** method to which we will pass the stream, as shown in the following code block:

```

public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
{
    if(value == null)
    {
        return null;
    }
    var bytes = (byte[])value;
    var stream = new MemoryStream(bytes);
    return ImageSource.FromStream(() => stream);
}

```

The view will contain the photo, which will take up two-thirds of the screen. Under the photo, we will add a description of the result. Let's set this up by going through the following steps:

1. In the **Views** folder, create a new file using the .NET MAUI **ContentPage** (XAML) file template, and name it **ResultView**.
2. Import the namespace for the converter.
3. Add **BytesToImageConverter** to **Resources** for the page and give it "ToImage" key.
4. Bind the **Title** property of **ContentPage** as the **Title** property of **ViewModel**.
5. Add **Grid** to the page with two rows. The **Height** value for the first **RowDefinition** should be **2***. The height of the second row should be *****. These are relative values that mean that the first row will take up two-thirds of **Grid**, while the second row will take up one-third of **Grid**.
6. Add **Image** to **Grid**, and bind the **Source** property to the **PhotoBytes** property in **ViewModel**. Use the converter to convert the bytes to an **ImageSource** object and set the **Source** property.
7. Add **Label**, and bind the **Text** property to the **Description** property of **ViewModel**, as shown in the following code block:

```

<ContentPage xmlns="http://xamarin.com/schemas/2014/forms" xmlns:x="http://schemas.microsoft.com
    namespace:HotdogOrNot.Converters"
    x:Class="HotdogOrNot.Views.ResultView" Title="{Binding Title}">
    <ContentPage.Resources>
        <converters:BytesToImageConverter x:Key="ToImage" />
    </ContentPage.Resources>
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="2*" />
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>
        <Image Source="{Binding PhotoBytes, Converter=
        {StaticResource ToImage}}" Aspect="AspectFill" />
        <Label Grid.Row="1" HorizontalOptions="Center" FontAttributes="Bold" Margin="10" Text="{Bin
    </Grid>
</ContentPage>

```

We also need to set **BindingContext** of the view. We will do this in the same way as we did in **MainView** – in the code-behind file (**ResultView.xaml.cs**), as shown in the following code snippet:


```
public ResultView (ResultViewModel viewModel)
{
    InitializeComponent ();
    BindingContext = viewModel;
}
```

We are now ready to write the initialization code for the app.

Initializing the app

We will set up **inversion of control (IoC)** and configure the main page for **Shell**.

Open **App.xaml.cs**, and set **MainPage** to **MainView** by following these steps:

1. Delete the **MainPage.xaml** and **MainPage.xaml.cs** files from the root of the project, since we won't be needing those.
2. Open the **AppShell.xaml** file in the root of the project, and modify it to look like the following code:

```
<?xml version="1.0" encoding="UTF-8" ?>
<Shell
    x:Class="HotdogOrNot.AppShell"
    xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:views="clr-namespace:HotdogOrNot.Views"
    Shell.FlyoutBehavior="Disabled">
    <ShellContent
        Title="Home"
        ContentTemplate="{DataTemplate views:MainView}"
        Route="MainView" />
</Shell>
```

Now, configure the **View** and **ViewModel** classes in the IoC container by following these steps:

1. Open the **MauiProgram.cs** file.
2. In the **CreateMauiApp** method before the **return** statement, add the following highlighted lines of code:

```
#if DEBUG
    builder.Logging.AddDebug();
#endif
builder.Services.AddTransient<Views.MainView>();
builder.Services.AddTransient<Views.ResultView>();
builder.Services.AddTransient<ViewModels.MainViewModel>();
builder.Services.AddTransient<ViewModels.ResultViewModel>();
return builder.Build();
```

The very last thing we need to do is add the route to **ResultView** to enable navigation from **MainView**. We will do this by adding the following highlighted code to the constructor of **AppShell** in **AppShell.xaml.cs**:

```
public AppShell()
{
    Routing.RegisterRoute("Result", typeof(HotdogOrNot.Views.ResultView));
    InitializeComponent();
}
```

Now, we are ready to run the app. If we use the simulator/emulator, we can just drag and drop photos to it if we need photos to test with. When the app has started, we can now pick a photo and run it against the model. The following screenshot shows how the app will look if we upload a photo of a hot dog:



Figure 12.13 – HotdogOrNot running in an Android emulator

NOTE

The prediction result for Android may not be as accurate as the web portal at <https://github.com/Azure-Samples/cognitive-services-android-customvision-sample/issues/12>. If you desire better, more consistent results, you can use the REST APIs.

Summary

In this chapter, we built an app that can recognize whether a photo contains a hot dog or not. We accomplished this by training a machine learning model for image classification, using Azure Cognitive Services and the Custom Vision service.

We exported models for ML.NET, and we learned how to use it in an MAUI app that targets iOS, Mac Catalyst, Windows, and Android. In the app, a user can take a photo or pick one from their photo library. This photo will be sent to the model to be classified, and we will get a result that tells us whether the photo is of a hot dog.

Now, we can continue to build other apps and use what we have learned in this chapter regarding machine learning, both on-device and in the cloud using Azure Cognitive Services. Even if we are building other apps, the concept will be the same.

Now, we have completed all the chapters in this book. We have learned the following:

- What .NET MAUI is and how we can get started building apps
- How to use the basic layouts and controls of .NET MAUI
- How to work with navigation
- How to make the user experience better with animations
- How to use sensors such as the **Global Positioning System (GPS)** in the background
- How to build apps for multiple form factors
- How to build real-time apps powered by Azure
- How to make apps smarter with machine learning

The next step is to start to build your own apps. To stay up to date and learn more about .NET MAUI, our recommendation is to read the official Microsoft dev blogs and watch live streams on Twitch and YouTube videos from the .NET MAUI team.

Thank you for reading the book!